Optimización de costos

Costo actual: **USD 0 / mes**

Servicio	Plan	Límite	Uso real del proyecto
Telegram Bot API	gratis	sin límite práctico	unos mensajes por día
Make	Free	1.000 operaciones/mes	~3-4 ops por registro/consulta; agregar desde el sitio suma otro ~3 (webhook + Gemini + Add a Row) → margen enorme
Google Gemini API (bot)	Free tier	límites por minuto y por día	1-2 llamadas por mensaje o por acción del sitio
Google Gemini API (CinefilIA)	Free tier, **proyecto aparte**	límites por minuto y por día, propios	2 llamadas por pregunta típica (pide una herramienta, después responde; hasta 5); ~2.000–3.500 tokens cada una
Google Sheets	gratis	10M celdas	~23.000 celdas (dos pestañas)
TMDB API	gratis	límite por IP, altísimo	la sincronización diaria (solo lo nuevo) + consultas en vivo desde el navegador de cada visitante
Vercel	Hobby (gratis)	100 GB/mes de tráfico; funciones con cupo mensual	trivial; 1 función (`/api/chat`)
GitHub Actions	gratis (repo público)	minutos ilimitados en repos públicos	1 corrida diaria de ~2-3 minutos
Total			**USD 0**

Estrategia por tarea

El principio: **modelo económico para lo mecánico, modelo capaz para el razonamiento, procesamiento por lotes para el volumen no urgente.**

Tarea	Naturaleza	Elección	Por qué
Clasificar intención + extraer ficha	Mecánico, repetitivo, alto volumen potencial	**Gemini Flash-Lite** (gratis)	Tarea acotada con esquema fijo; un modelo chico la resuelve igual de bien.
Responder consultas / recomendar (Telegram)	Razonamiento sobre ~1.300 filas de contexto	**Gemini Flash-Lite** con ventana larga	Alcanza para listar y recomendar.

Tarea	Naturaleza	Elección	Por qué
Chat de cine del sitio (CinefilIA)	Conversación, en tiempo real, abierta al público	**Gemini Flash-Lite** + controles del lado del servidor	Ver "El caso CinefilIA" abajo: lo barato y rápido, compensado con verificación propia en vez de un modelo más caro.
Pósters, reparto, duración, sinopsis, recomendadores, perfil de gusto (~1.250 películas c/u)	Volumen alto, **no urgente**	**GitHub Actions + scripts + TMDB**, NO Make	~1.250 llamadas por Make = más que el límite mensual entero, por cada archivo. Un script lo hace en minutos, gratis; corriendo solo de noche, además no depende de nadie.
Completar director/año/país/género al agregar **un** título desde el sitio	Puntual, un solo ítem por vez	**Gemini vía Make**, sí	Acá sí tiene sentido: es 1 llamada, en tiempo real, no un backfill.
Búsquedas en vivo (persona, país, póster que falta)	Puntual, la pide un visitante	**TMDB directo desde el navegador**	Sin servidor ni IA: es una consulta a una API gratuita de solo lectura.

El caso CinefilIA

El chat del sitio es la única pieza abierta al público que llama a una IA, así que es donde más importa el costo (aunque sea "cupo gratis", se agota).

- **De mandar todo a pedir lo justo.** La primera versión mandaba el catálogo entero con todo el detalle en cada pregunta: ~~29.000 tokens, y a las ~6 preguntas seguidas agotaba el cupo gratis por minuto. La segunda mandaba el detalle solo de lo relacionado con la pregunta: ~10.000. La actual es un **agente con herramientas**: el modelo recibe la pregunta y pide los datos que necesita ("buscar_vistas director=Scorsese"), así que cada pedido lleva **~~2.000–3.500 tokens**. Son 2 llamadas en vez de 1, pero en total ~5.000 tokens: la mitad que antes y un sexto de la primera versión. Medido: ~1,5–4 s por respuesta (el plan gratis a veces tarda bastante más; hay un tope de 30 s).
- **Cuentas hechas por código, no por la IA.** Las herramientas devuelven totales y listas ya contados. Más barato (menos razonamiento) y correcto (el modelo sumaba mal: decía 20 visionados de Tarantino y son 25; y completaba listas de memoria: Birdman para Emma Stone).
- **Sin base vectorial.** Las preguntas son de filtrar y contar (director, año, género), no de "buscar por parecido de texto", y el catálogo entra en memoria (~1.200 películas): con herramientas sobre datos estructurados alcanza y da resultados exactos. Para "parecidas" se usa TMDB.
- **Verificación en vez de un modelo más caro.** El modelo rápido a veces recomienda algo ya visto. En lugar de pasar a un modelo mayor (que respondía mejor pero tardaba ~30 segundos), la función revisa cada recomendación contra el catálogo real y repite la consulta solo si hace falta.
- **Límites para un sitio público.** 15 consultas cada 10 minutos por persona, 600 caracteres por mensaje, las últimas 10 intervenciones de la charla. Y la clave en un **proyecto de Google aparte**: si alguien agota el cupo del chat, el bot de Telegram sigue funcionando.
- **Fuera de tema, en una línea.** Lo que no es de cine recibe una respuesta fija y corta, sin gastar tokens en contestar.

Qué NO hacer

- **No** mandar los backfills por Make. Un backfill de 1.000+ ítems agota el plan Free en una sola corrida. Van por la sincronización diaria.
- **No** llamar a la IA para lo que resuelve una fórmula o un script. `id`, `numero` y `es_revisionado` se calculan con `COUNTIF` en la hoja; los conteos de CinefilIA, en la función.
- **No** poner la clave de una IA en el navegador. La de TMDB sí está en el sitio (es de solo lectura y gratuita); la de Gemini vive en Vercel y el sitio solo habla con la función.

Si el proyecto escalara

Disparador	Qué cambiar
> 1.000 ops/mes en Make	Plan Core de Make (~~USD 9/mes, 10.000 ops~~) ~~o migrar la orquestación a n8n self-hosted~~ (USD 5-10/mes de VPS).
Se supera la cuota gratis de Gemini	Pago por uso de Gemini Flash-Lite: fracciones de centavo por mensaje.
Mucho tráfico en CinefilIA	Cache de respuestas frecuentes; bajar el límite por persona; o mover la detección de "fuera de tema" a una regla barata antes de llamar a la IA.
Consultas con mucho más volumen de catálogo	Embeddings + búsqueda por similitud → se le pasa a la IA solo el top-k, no la lista de títulos completa.
Se quiere calidad de razonamiento superior	Solo la rama de consultas a un modelo mayor (Gemini Pro / Claude); la de registro sigue en Flash-Lite.